ASSIGNMENT 3-CO3

NAME:THARANI RB

REG NO:1911260138

COURSE CODE:CSA0822

SLOT:B

GITHUB: GitHub - TharaniRB15/CSA0822-1911260138-

TASK 1-Lists and Slicing: Daily Rainfall Records

AIM:

To write a Python program to store daily rainfall readings in a list, calculate weekly totals, find the wettest and driest weeks, calculate a 7-day moving average, and identify dry days.

THEORY:

- List: Stores multiple rainfall readings in a single variable.
- Slicing: Extracts a portion of a list using index positions.
- Append(): Adds a new element to the end of a list.
- Max(): Finds the highest rainfall value.
- Min(): Finds the lowest rainfall value.

PROGRAM:

```python
import random
readings = [random.randint(0, 50) for i in range(365)]
print("Total Days:", len(readings))
weekly_totals = []
for i in range(0, 365, 7):
    week = readings[i:i+7]
    weekly_totals.append(sum(week))
print("\nWeekly Totals:", weekly_totals)
print("\nWettest Week:", weekly_totals.index(max(weekly_totals)) + 1)
print("Driest Week:", weekly_totals.index(min(weekly_totals)) + 1
print("\nMaximum Rainfall:", max(readings))
print("Minimum Rainfall:", min(readings))
moving_avg = []
for i in range(len(readings) - 6):
    avg = sum(readings[i:i+7]) / 7
    moving_avg.append(avg)
print("\n7-Day Moving Average:", moving_avg[:10])
dry_days = []
for i in range(len(readings)):
    if readings[i] == 0:
        dry_days.append(i + 1)
print("\nDry Days:", dry_days)
print("Number of Dry Days:", len(dry_days))
import time
start = time.perf_counter()
for i in range(len(readings) - 6):
    avg = sum(readings[i:i+7]) / 7
naive_time = time.perf_counter() - start
start = time.perf_counter()
running_avg = []
total = sum(readings[:7])
running_avg.append(total / 7)
for i in range(7, len(readings)):
    total = total - readings[i-7] + readings[i]
    running_avg.append(total / 7)
running_time = time.perf_counter() - start
print("\nNaive Time:", naive time)
```

OUTPUT:

```
====== RESTART: C:/Users/thara/OneDrive/Attachments/python practise/ba.py ======
First 10 Rainfall Readings: [5, 7, 19, 36, 43, 18, 41, 32, 50, 48]

Weekly Totals: [169, 219, 166, 192, 192, 157, 153, 205, 253, 191, 153, 159, 185, 162, 203, 170, 209, 17
238, 114, 183, 157, 191, 180, 137, 163, 198, 220, 279, 202, 116, 206, 194, 164, 172, 194, 2]

Maximum Rainfall: 50
Maximum Day: 9
Minimum Rainfall: 0
Minimum Day: 38

Dry Days: [38, 39, 80, 97, 210]

Naive Average: [24.142857142857142, 28.0, 34.142857142857146, 38.285714285714285, 34.142857142857146]
Running Average: [24.142857142857142, 28.0, 34.142857142857146, 38.285714285714285, 34.142857142857146]

Readings: 365
Naive Time: 0.00016789999790489674
Running Sum Time: 7.920002099126577e-05

Readings: 36500
Naive Time: 0.01814090000698343
Running Sum Time: 0.007184399990364909

Readings: 365000
Naive Time: 0.18001290003303438
Running Sum Time: 0.08962600002996624

Empty List: []
Short List: []
```

TASK 2-Tuples, Mutability and Copying

AIM:

To write a Python program using tuples, lists, and dictionary keys to create a bus-stop zone grid and demonstrate aliasing, shallow copying, deep copying, and tuple immutability.

THEORY:

- Tuple: An ordered and immutable collection of elements.
- List: A mutable collection that allows adding, removing, and modifying elements.
- Aliasing: Two variables referring to the same list in memory.
- Shallow Copy: Creates a new list but shares nested objects.
- Deep Copy: Creates a completely independent copy of nested objects.

PROGRAM:

File Edit Format Run Options Window Help

```
import copy
import sys
import time
rows, cols = 3, 4
bad = [[0] * cols] * rows
bad[0][1] = 7
good = [[0 for j in range(cols)] for i in range(rows)]
good[0][1] = 7
print("Bad Grid:", bad)
print("Good Grid:", good)
print("Bad rows same:", bad[0] is bad[1])
print("Good rows same:", good[0] is good[1])
stops = {
    (13.0827, 80.2707): "Chennai Central",
    (13.0418, 80.2341): "T. Nagar"}
print("\nBus Stops:")
print(stops[(13.0418, 80.2341)])
print("\nTuple Unpacking:")
for (lat, lon), name in stops.items():
    print(name, "Latitude:", lat, "Longitude:", lon)
original = [[1, 2], [3, 4]]
shallow = copy.copy(original)
deep = copy.deepcopy(original)
original[0][0] = 99
print("\nOriginal:", original)
print("Shallow Copy:", shallow)
print("Deep Copy:", deep)
sliced = original[:]
print("Sliced Copy:", sliced)
my_list = list(range(1000000))
my_tuple = tuple(range(1000000))
print("\nList Memory:", sys.getsizeof(my_list))
print("Tuple Memory:", sys.getsizeof(my_tuple))
start = time.perf_counter()
for i in range(1000):
    x = my_list[i]
list_time = time.perf_counter() - start
start = time.perf_counter()
for i in range(1000):
    x = my_tuple[i]
tuple_time = time.perf_counter() - start
print("\nList Time:", list_time)
print("Tuple Time:", tuple_time)
data = ([10, 20], 30)
data[0].append(40)
print("\nTuple with List:", data)
```

OUTPUT:

```
==================================================
Bad Grid: [[0, 7, 0, 0], [0, 7, 0, 0], [0, 7, 0, 0]]
Good Grid: [[0, 7, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]]
Bad rows same: True
Good rows same: False

Bus Stops:
T. Nagar

Tuple Unpacking:
Chennai Central Latitude: 13.0827 Longitude: 80.2707
T. Nagar Latitude: 13.0418 Longitude: 80.2341

Original: [[99, 2], [3, 4]]
Shallow Copy: [[99, 2], [3, 4]]
Deep Copy: [[1, 2], [3, 4]]
Sliced Copy: [[99, 2], [3, 4]]

List Memory: 8000056
Tuple Memory: 8000048

List Time: 0.00026119999998407555
Tuple Time: 0.00027510000001808279

Tuple with List: ([10, 20, 40], 30)
```

TASK 3-Dictionaries: Farmer Helpline Complaint Analysis

AIM:

To write a Python program using dictionaries to count frequently used words in farmer complaints, group complaints by district, identify top words, and remove common words using a stop-word set.

THEORY:

- Dictionary: Stores data in key-value pairs.
- get(): Retrieves a value from a dictionary without causing an error if the key is missing.
- setdefault(): Creates a key with a default value if it does not exist.
- keys(): Returns all dictionary keys.
- values(): Returns all dictionary values.

PROGRAM:

```
import re
import time
from collections import Counter
calls = ["seeds not received seeds delayed","fertiliser price hi
    "seeds price high subsidy not received",
    "water problem in farm",
    "fertiliser not received"]
STOP = {"not", "high", "in", "the", "and"}
def frequencies(texts, stop):
    freq = {}
    for text in texts:
        words = re.findall(r"[a-z]+", text.lower())
        for word in words:
            if word not in stop:
                freq[word] = freq.get(word, 0) + 1
    return freq
def group_district(data):
    district = {}
    for name, complaint in data:
        district.setdefault(name, []).append(complaint)
    return district
def top_words(freq, k):
    return sorted(freq.items(), key=lambda x: (-x[1], x[0]))[:k]
complaint_data = [("Chennai", "seeds not received"),
    ("Madurai", "fertiliser price high"),
    ("Chennai", "seeds delayed"),
    ("Salem", "water problem"),
    ("Madurai", "seeds price high")]
freq = frequencies(calls, STOP)
print("Word Frequencies:")
print(freq)
print("\nTop 3 Words:")
print(top_words(freq, 3))
print("\nDistrict Complaints:")
print(group_district(complaint_data))
words = [word for text in calls
         for word in re.findall(r"[a-z]+", text.lower())
         if word not in STOP]
start = time.perf_counter()
dictionary_result = frequencies(calls, STOP)
dict_time = time.perf_counter() - start
start = time.perf_counter()
list_result = {
    word: words.count(word) for word in set(words)}
list_time = time.perf_counter() - start
print("\nDictionary Time:", dict_time)
print("List Count Time:", list_time)
```

OUTPUT:

```
====== RESTART: C:/Users/thara/OneDrive/Attachments/python practise/ba.py ======
Word Frequencies:
{'seeds': 3, 'received': 3, 'delayed': 1, 'fertiliser': 2, 'price': 2, 'subsidy'
: 1, 'water': 1, 'problem': 1, 'farm': 1}

Top 3 Words:
[('received', 3), ('seeds', 3), ('fertiliser', 2)]

District Complaints:
{'Chennai': ['seeds not received', 'seeds delayed'], 'Madurai': ['fertiliser pri
ce high', 'seeds price high'], 'Salem': ['water problem']}

Dictionary Time: 2.5700000151111873e-05
List Count Time: 1.5399999938381 37e-05

Counter Result:
Counter({'seeds': 3, 'received': 3, 'fertiliser': 2, 'price': 2, 'delayed': 1, '
subsidy': 1, 'water': 1, 'problem': 1, 'farm': 1})
```

TASK 4-Sets: Elective Registration Clash Detection

AIM:

To write a Python program using sets to identify students registered in both electives, exactly one elective, and neither elective using set operations.

THEORY:

- Set: An unordered collection of unique elements.
- Union (|): Combines all elements from two sets.
- Intersection (&): Finds common elements in both sets.
- Difference (-): Finds elements present in one set but not in another.
- Symmetric Difference (^): Finds elements present in exactly one set.
- Membership (in): Checks whether an element exists in a set.
- Duplicate Removal: Sets automatically remove duplicate elements

PROGRAM:

File Edit Format Run Options Window Help

```
ai_elective = {"192311001", "192311004", "192311007", "192311009"}
iot_elective = {"192311002", "192311004", "192311009", "192311010"}
all_students = {f"192311{i:03d}" for i in range(1, 11)}
both = ai_elective & iot_elective
exactly_one = ai_elective ^ iot_elective
neither = all_students - (ai_elective | iot_elective)
union = ai_elective | iot_elective
difference = ai_elective - iot_elective
print("Clash       :", sorted(both))
print("Exactly one :", sorted(exactly_one))
print("Neither     :", sorted(neither))
print("Union       :", sorted(union))
print("Difference  :", sorted(difference))
students = ["101", "102", "101", "103"]
unique_students = set(students)
print("Unique Students:", sorted(unique_students))
print("Student 192311004 registered:",
      "192311004" in ai_elective)
```

OUTPUT:

```
====== RESTART: C:/Users/thara/OneDrive/Attachments/python practise/ba.py ======
Clash       : ['192311004', '192311009']
Exactly one : ['192311001', '192311002', '192311007', '192311010']
Neither     : ['192311003', '192311005', '192311006', '192311008']
Union       : ['192311001', '192311002', '192311004', '192311007', '192311009', '192311010']
Difference  : ['192311001', '192311007']
Unique Students: ['101', '102', '103']
Student 192311004 registered: True
```

TASK 5-List Comprehension: Cleaning Sensor Readings

AIM:

To write a Python program to clean and convert cold-storage temperature readings into Celsius and Fahrenheit using for loops, list comprehension, map(), and filter(), and identify alarm minutes.

THEORY:

- List Comprehension: Creates a new list using a short syntax with conditions.
- For Loop: Repeats statements for each reading.
- Append(): Adds converted values to a list.
- Map(): Applies a function to every element.
- Filter(): Selects elements based on a condition.

PROGRAM:

File Edit Format Run Options Window Help

```
import time
import sys
readings = [4, 5, 7, 9, 2, 50, -50, 6, 8, 3]
valid = []
for i in readings:
    if -50 <= i <= 50:
        valid.append(i)
celsius = [i for i in valid]
fahrenheit = [(i * 9/5) + 32 for i in valid]
print("Valid Celsius:", celsius)
print("Fahrenheit:", fahrenheit)
celsius_map = list(map(lambda x: x, filter(lambda x: -50 <= x <= 50, readings)))
print("Using Map and Filter:", celsius_map)
alarms = [i for i, x in enumerate(readings) if x < 2 or x > 8]
print("Alarm Minutes:", alarms)
start = time.perf_counter()
result1 = [x for x in readings if -50 <= x <= 50]
time1 = time.perf_counter() - start
start = time.perf_counter()
result2 = list(filter(lambda x: -50 <= x <= 50, readings))
time2 = time.perf_counter() - start
print("List Comprehension Time:", time1)
print("Map and Filter Time:", time2)
generator = (x for x in readings if -50 <= x <= 50)
print("Generator Memory:", sys.getsizeof(generator))
print("List Memory:", sys.getsizeof(result1))
```

OUTPUT:

```
====== RESTART: C:/Users/thara/OneDrive/Attachments/python practise/ba.py
Valid Celsius: [4, 5, 7, 9, 2, 50, -50, 6, 8, 3]
Fahrenheit: [39.2, 41.0, 44.6, 48.2, 35.6, 122.0, -58.0, 42.8, 46.4, 37.4]
Using Map and Filter: [4, 5, 7, 9, 2, 50, -50, 6, 8, 3]
Alarm Minutes: [3, 5, 6]
List Comprehension Time: 9.499999578110874e-06
Map and Filter Time: 1.4899999769113492e-05
Generator Memory: 216
List Memory: 184
```

TASK 6-Nested Data: A Railway Timetable

AIM:

To write a Python program using nested dictionaries, lists, tuples, and sets to maintain a railway timetable, search trains by station and day, sort trains by departure time, and create an index for faster searching.

THEORY:

- Dictionary: Stores train details using key-value pairs.
- List: Stores multiple station details of a train.
- Tuple: Stores station name, arrival time, and departure time.
- Set: Stores the running days of each train without duplicates.
- Sorted(): Arranges trains in ascending order.
- Lambda: Provides a key for sorting train departure times.

PROGRAM:

```
trains={
"101":{"name":"Chennai Express","stops":[("Chennai","06:00","06:15"),("Katpadi",
"102":{"name":"Pandian Express","stops":[("Madurai","07:00","07:15"),("Trichy","
"103":{"name":"Coimbatore Express","stops":[("Chennai","05:00","05:10"),("Salem"
"104":{"name":"Vaigai Express","stops":[("Chennai","06:30","06:45"),("Villupuram
"105":{"name":"Bangalore Express","stops":[("Chennai","07:00","07:15"),("Katpadi
}
station_index={}
for number,details in trains.items():
    for stop in details["stops"]:
        station_index.setdefault(stop[0],[]).append(number)
station=input("Enter station: ").title()
day=input("Enter day: ").capitalize()
print("\nAvailable Trains:")
available=[]
for number in station_index.get(station,[]):
    if day in trains[number]["days"]:
        available.append((number,trains[number]["name"],next(stop[2] for stop in
available=sorted(available,key=lambda x:x[2])
for number,name,departure in available:
    print(number,name,"Departure:",departure)
print("\nComplete Railway Timetable:")
for number,details in trains.items():
    print("\nTrain:",number,details["name"])
    print("Running Days:",sorted(details["days"]))
    for station_name,arrival,departure in details["stops"]:
        print(station_name,"Arrival:",arrival,"Departure:",departure)
print("\nStation Index:")
print(station_index)
```

OUTPUT:



TASK 7-Design a Contact-Merging Program from First Principles

AIM:

To develop a Python program to merge contacts from multiple phones and a SIM card by removing duplicate contacts using normalized phone numbers, dictionaries, and sets.

THEORY:

- Dictionary: Stores contacts using normalized phone numbers as keys.
- Set: Stores unique phone numbers and email addresses.
- Normalization: Converts names and phone numbers into a standard format.
- Strip(): Removes unwanted spaces.
- Replace(): Removes spaces and special characters.

PROGRAM:

```
import re
import time
def normalize_name(name):
    return " ".join(name.lower().strip().split())
def normalize_number(number):
    number = re.sub(r"\D", "", number)
    if len(number) == 10:
        number = "91" + number
    return number
def merge_contacts(contacts):
    merged = {}
    for contact in contacts:
        name = normalize_name(contact.get("name", "Unknown"))
        number = normalize_number(contact.get("phone", ""))
        email = contact.get("email", "").lower().strip()
        if not number:
            continue
        if number not in merged:

                merged[number] ={"name": contact.get("name", "").strip(),
                "numbers": set(),
                "emails": set()}
        merged[number]["numbers"].add(number)
        if email:
            merged[number]["emails"].add(email)
    return merged
contacts = [
    {"name": "Dr. Ravi", "phone": "+91 9840012345", "email": "ravi@gmail.com"},
    {"name": "Ravi Kumar", "phone": "9840012345", "email": "ravi@gmail.com"},
    {"name": "Dr. Ravi", "phone": "9840012346", "email": "ravi@yahoo.com"},
    {"name": "Arun", "phone": "9876543210", "email": "arun@gmail.com"},
    {"name": "Arun Kumar", "phone": "+91 9876543210", "email": "arun@yahoo.com"}]
start = time.perf_counter()
result = merge_contacts(contacts)
end = time.perf_counter()
print("Merged Contacts:")
for number, details in result.items():
    print("\nName:", details["name"])
    print("Numbers:", sorted(details["numbers"]))
    print("Emails:", sorted(details["emails"]))
print("\nTotal Contacts:", len(result))
print("Execution Time:", end - start)
```

OUTPUT:

```
====== RESTART: C:/Users/thara/OneDrive/Atta
Merged Contacts:

Name: Dr. Ravi
Numbers: ['919840012345']
Emails: ['ravi@gmail.com']

Name: Dr. Ravi
Numbers: ['919840012346']
Emails: ['ravi@yahoo.com']

Name: Arun
Numbers: ['919876543210']
Emails: ['arun@gmail.com', 'arun@yahoo.com']

Total Contacts: 3
Execution Time: 0.0002540999943803996
```